



MCQ

1. Which keyword in Java is used to explicitly generate an exception?

- a) throws
 - b) throw
 - c) final
 - d) error
-

2. The object after the throw keyword must be:

- a) Only Exception
 - b) Only RuntimeException
 - c) Any Throwable subclass object
 - d) Only Error
-

3. What is the correct syntax?

- a) throw new ArithmeticException
 - b) throw Exception("error")
 - c) throw new Exception("msg")
 - d) throws new Exception("msg")
-

4. Which of the following is TRUE about the throw keyword?

- a) Can throw multiple exceptions at once
- b) Can throw only one exception at a time
- c) Cannot throw custom exceptions
- d) Must be written in method signature



CJC

Complete Java Classes

by **Kunal Sir**

5. Custom exceptions must extend:

- a) Object
- b) Throwable
- c) ArithmeticException
- d) IOException

6. Where can the throw keyword appear?

- a) Inside method body
- b) With method signature
- c) Outside class
- d) Only in constructors

7. Which of the following is an UNCHECKED exception?

- a) IOException
- b) FileNotFoundException
- c) ArithmeticException
- d) SQLException

8. What happens after a throw statement executes?

- a) Program continues
 - b) Remaining code inside method is skipped
 - c) Only loop stops
 - d) Exception is suppressed
-



CJC

Complete Java Classes

by Kunal Sir

9. What is the output?

```
throw new NullPointerException("Error");  
System.out.println("Hello");
```

- a) Prints Error then Hello
 - b) Prints Hello
 - c) Compilation error
 - d) Only throws exception, Hello not printed
-

10. throw keyword can throw:

- a) Only pre-defined exceptions
 - b) Only user-defined exceptions
 - c) Both user-defined & predefined exceptions
 - d) None
-

11. What type of exception is thrown using:

```
throw new RuntimeException();
```

- a) Checked
 - b) Unchecked
 - c) Both
 - d) None
-

12. Which must be TRUE for a valid throw?

- a) Object must be Throwable
 - b) Object must be Error only
 - c) Object must be Exception only
 - d) No object required
-



CJC

Complete Java Classes

by Kunal Sir

13. throw is used for:

- a) Handling exception
 - b) Declaring exception
 - c) Explicitly throwing exception
 - d) Stopping JVM
-

14. After using throw, execution goes to:

- a) next line
 - b) catch block
 - c) finally always
 - d) compiler
-

15. Which scenario best uses throw?

- a) Database operations
 - b) Input validation failures
 - c) Multithreading
 - d) File reading
-

16. What is thrown?

```
throw new Throwable();
```

- a) Compilation error
 - b) Valid – Throwable allowed
 - c) Valid only inside try
 - d) JVM error
-



CJC

Complete Java Classes

by **Kunal Sir**

17. What is required BEFORE using throw for a CHECKED exception?

- a) Nothing
 - b) Must handle or declare
 - c) JVM handles automatically
 - d) Must extend RuntimeException
-

18. Which exception is UNCHECKED?

- a) NullPointerException
 - b) IOException
 - c) SQLException
 - d) ClassNotFoundException
-

19. Can you write multiple throw statements in one method?

- a) Yes
 - b) No
 - c) Yes, but only one will execute
 - d) Only inside loops
-

20. The main purpose of throw is:

- a) Propagation
 - b) Explicit exception generation
 - c) Error logging
 - d) Thread stopping
-
-



CJC

Complete Java Classes

by Kunal Sir

21. The throws keyword is used for:

- a) Handling exception
 - b) Declaring exception
 - c) Logging exception
 - d) Creating custom exceptions
-

22. throws is written:

- a) Inside method
 - b) Outside class
 - c) With method signature
 - d) Only in constructors
-

23. Which keyword gives the CALLER a chance to handle an exception?

- a) try
 - b) throw
 - c) finally
 - d) throws
-

24. What is the correct syntax?

- a) void test() throw Exception
 - b) void test() throws Exception
 - c) void test() throw new Exception
 - d) throw test()
-



CJC

Complete Java Classes

by **Kunal Sir**

25. throws can declare how many exceptions?

- a) Only one
 - b) Only two
 - c) Multiple
 - d) None
-

26. Which exception MUST be declared using throws?

- a) ArithmeticException
 - b) NullPointerException
 - c) IOException
 - d) ArrayIndexOutOfBoundsException
-

27. Which is TRUE about throws?

- a) Mandatory only for unchecked exceptions
 - b) Mandatory only for checked exceptions
 - c) Mandatory for all exceptions
 - d) Never mandatory
-

28. What does this mean?

`void test() throws IOException`

- a) test() will always throw exception
 - b) test() may throw IOException
 - c) test() handles IOException
 - d) None
-



CJC

Complete Java Classes

by Kunal Sir

29. If a method declares throws, whose responsibility is it to handle the exception?

- a) Callee
 - b) Caller
 - c) JVM
 - d) Compiler
-

30. Which is not allowed?

- a) throws IOException, SQLException
 - b) throws IOException
 - c) throws new IOException()
 - d) throws Exception
-

31. Can constructors use throws?

- a) Yes
 - b) No
-

32. Which keyword reduces the need for multiple try-catch blocks?

- a) throw
 - b) throws
 - c) catch
 - d) final
-



CJC

Complete Java Classes

by **Kunal Sir**

33. What is required when calling a method containing a CHECKED exception without handling it?

- a) nothing
 - b) declare same exception using throws
 - c) JVM auto-handles
 - d) remove exception
-

34. throws keyword helps in:

- a) Exception propagation
 - b) Exception handling
 - c) Exception suppression
 - d) None
-

35. Which is UNCHECKED?

- a) Exception
 - b) IOException
 - c) RuntimeException
 - d) FileNotFoundException
-

36. If a method uses throws, does it guarantee exception will be thrown?

- a) Yes
 - b) No
-



CJC

Complete Java Classes

by Kunal Sir

37. What happens if a CHECKED exception is neither caught nor declared?

- a) Runtime error
 - b) Compilation error
 - c) JVM ignores
 - d) None
-

38. Which is NOT propagated using throws?

- a) IOException
 - b) SQLException
 - c) RuntimeException
 - d) All can be propagated
-

39. throws is followed by:

- a) Exception object
 - b) Exception class
 - c) Error instance
 - d) Throwable object
-

40. Advantages of throws?

- a) Cleaner code
 - b) Delegates exception handling
 - c) Avoids unnecessary try-catch
 - d) All
-



Practical Questions

Q1. Input Age Validation – Custom Exception

Context: A website needs to restrict access to adult-only content.

Task: Build a console app that takes an integer `age` from the user and validates it.

Requirements

- Define a checked custom exception class `UnderAgeException` extends `Exception` with:
 - Constructors: default and one that accepts a message.
 - A meaningful message (e.g., "Age must be 18 or above.").
- Read `age` from standard input.
- If `age < 18`, **use `throw`** to throw `UnderAgeException`.
- If valid, print "Access granted".

Constraints

- Input must be numeric (handle non-numeric inputs gracefully with a friendly message without crashing).
- Only **one** exception is thrown at a time via `throw`.

I/O Format

- **Input:** Single integer (e.g., 17)
- **Output:** Either the exception message or "Access granted".

Edge Cases

- `age = 18` (boundary).
- Negative ages or extremely large values.
- Empty input / whitespace.

Acceptance Criteria



by Kunal Sir

- Custom exception defined and thrown using `throw`.
- Program skips any statements after `throw` inside the same flow.
- Clear, user-friendly output.

Q2. Bank Withdrawal – ArithmeticException

Context: Simulate a basic ATM withdrawal.

Task: Implement balance checking and raise an exception when funds are insufficient.

Requirements

- **Variables:** `double balance`, `double amount`.
- If `amount > balance`, **throw** `new ArithmeticException("Insufficient Funds")`.
- Otherwise deduct and print the remaining balance.

Constraints

- Amount and balance must be non-negative.
- Decimal values allowed.

I/O Format

- **Input:** Two numbers on separate lines: balance, amount.
- **Output:** Either "Remaining balance: X" or exception message.

Edge Cases

- `amount == balance` (should become zero).
- `amount = 0`.
- Very large amounts, floating rounding.

Acceptance Criteria

- Exception is thrown *explicitly* via `throw`.
- No code after the `throw` in the same block executes.



Q3. Username Check – InvalidUsernameException

Context: Enforce username rules in a signup form.

Task: Reject usernames containing digits.

Requirements

- Create `InvalidUsernameException` extends `Exception` (checked).
- Input: a single `String` username.
- If username contains any digit (0-9), **throw** `InvalidUsernameException("Username must contain only letters.")`.
- On success, print "Username accepted".

Constraints

- Empty or whitespace-only usernames should also be rejected with a **different** message (e.g., "Username cannot be empty"). You can use a simple `if` and `throw`.
- Case-insensitive letters allowed, no spaces inside.

I/O Format

- **Input:** A single line (username)
- **Output:** Acceptance message or exception message.

Edge Cases

- "john123", "", " ", "John_Doe" (decide underscore policy and document it).

Acceptance Criteria

- Uses pattern check (e.g., regex) or character scan.
- Throws **only one** exception per invalid entry.



Q4. Division Program – Zero Denominator

Context: Safe math utility for division.

Task: Divide two integers and prevent division by zero.

Requirements

- Read two ints: numerator, denominator.
- If denominator == 0, **throw** new `ArithmeticException("Cannot divide by zero")`.
- Else print the integer quotient and remainder.

Constraints

- Use integer division.
- Print in the format: "Quotient: q, Remainder: r".

I/O Format

- **Input:** Two integers
- **Output:** Result line or exception message.

Edge Cases

- Negative values.
- Very large or very small numbers (int range).
- 0 / nonZero.

Acceptance Criteria

- Uses `throw` for the zero case.
- No further computation after throwing.



Q5. Voting Eligibility – validateVote(int age)

Context: Election commission validation.

Task: Implement a reusable validator.

Requirements

- Define class `VotingException` extends `Exception` (checked).
- Method: `static void validateVote(int age):`
 - If `age < 18`, **throw** new `VotingException("Not eligible to vote")`.
 - Else print "Eligible to vote".
- In main, read age and call `validateVote`.

Constraints

- `validateVote` must contain the `throw`.
- `main` may either catch it or let the JVM show the stack trace (your choice—state which you implement).

I/O Format

- **Input:** One integer
- **Output:** Either eligibility text or exception message.

Edge Cases

- `age = 17, 18, negative numbers`.

Acceptance Criteria

- Correct use of custom checked exception + `throw`.
- Clear separation of validation logic in a method.



Throws Keyword Problems

Problem 1: Division Utility Method

Concepts Used: Methods, integer division, checked exception, throws.

Difficulty: Easy

Problem Statement:

Create a method `divide(int a, int b)` that performs integer division and **declares**:

`divide(int a, int b) throws ArithmeticException`

Inside the method:

- If `b == 0`, **throw** new `ArithmeticException("Cannot divide by zero")`.
- Otherwise return the quotient.

In main, call the method and handle the exception using try-catch.

Problem 2: Age Validator Method

Concepts Used: Methods, custom exception, throws.

Difficulty: Medium

Problem Statement:

Define a custom exception class `AgeException` (extends `Exception`).

Write a method:

`static void validateAge(int age) throws AgeException`

Inside:



by Kunal Sir

- If `age < 18`, throw `AgeException("Age must be 18 or above")`.

In `main`, read age from user and call the method.
Use try-catch in `main` to print proper message.

Problem 3: Marks Validation

Concepts Used: Basic loops/conditions, throws, custom checked exception.

Difficulty: Medium

Problem Statement:

Create a custom exception `InvalidMarksException`.

Write a method:

```
static void checkMarks(int marks) throws InvalidMarksException
```

Inside the method:

- If `marks < 0` or `marks > 100`, throw the custom exception.

In `main`, call the method with different sample inputs and print whether marks are valid.

✓ Problem 4: Login Password Validation

Concepts Used: String handling, methods, throws, custom exception.

Difficulty: Medium

Problem Statement:

Create a custom exception `WeakPasswordException`.

Write a method:



CJC

Complete Java Classes

by Kunal Sir

```
static void validatePassword(String pwd) throws WeakPasswordException
```

Conditions for valid password (only simple rules—core Java level):

- Length must be ≥ 6 .
- Must **not** contain spaces.

If invalid, throw `WeakPasswordException`.

Call the method from main and handle it with try-catch.

Problem 5: Array Index Checker

Concepts Used: Arrays, length checking, throws, built-in exception.

Difficulty: Easy

Problem Statement:

Write a method:

```
static int getValue(int[] arr, int index) throws  
ArrayIndexOutOfBoundsException
```

If index is not valid ($\text{index} < 0$ or $\text{index} \geq \text{arr.length}$), throw the exception.

In main, create an array and test valid/invalid indexes.



Problem 6: Even Number Checker

Concepts Used: Basic conditions, throws, custom exception.

Difficulty: Easy

Problem Statement:

Create a custom exception `OddNumberException`.

Write a method:

```
static void checkEven(int num) throws OddNumberException
```

If num is odd, throw `OddNumberException("Number must be even")`.

Call the method from main.

Problem 7: Simple Calculator

Concepts Used: Switch-case, methods, throws, basic arithmetic.

Difficulty: Medium

Problem Statement:

Create a method:

```
static int calculate(int a, int b, char op) throws ArithmeticException,  
Exception
```

Rules:

- If op is +, -, *, /, perform operation.
- If op is / and $b == 0 \rightarrow$ throw `ArithmeticException`.
- If op is anything else $\rightarrow$ throw `Exception("Invalid operator")`.



Call method from main and handle both exceptions.

Problem 8: Factorial Calculator

Concepts Used: Loop, maths, throws, custom exception.

Difficulty: Medium

Problem Statement:

Create a custom exception `NegativeNumberException`.

Write a method:

```
static int factorial(int n) throws NegativeNumberException
```

If $n < 0$, throw the exception.

Otherwise compute factorial using a loop.

Problem 9: Student Grade Checker

Concepts Used: Condition + method + throws.

Difficulty: Simple

Problem Statement:

Write a method:

```
static char getGrade(int percentage) throws Exception
```

Rules:

- If $\text{percentage} < 0$ or $> 100 \rightarrow$ throw `Exception("Invalid percentage")`.
- Otherwise:



CJC

Complete Java Classes

by Kunal Sir

- = 90 → 'A'
- = 75 → 'B'
- = 50 → 'C'
- else → 'D'

Problem 10: Number Range Validator

Concepts Used: Simple comparisons, throws, custom exception.

Difficulty: Easy

Problem Statement:

Create custom exception `OutOfRangeException`.

Write a method:

```
static void checkRange(int num) throws OutOfRangeException
```

If number is not between 1 and 100 (inclusive), throw exception.

Call from main and show the error message.